



ONE OBJECTIVE. YOUR AGENTS, WORKING TOGETHER.

The complete beginner guide

Start from zero. Understand the product, prepare your Mac, create your agents, and follow eight practical use cases one step at a time.

-
- 01** Understand the ideas and the screens
 - 02** Set up models, browsers, files, and accounts
 - 03** Create an agent and run your first task
 - 04** Try eight worked use-case walkthroughs

Local desktop pre-release, version 0.10.0.

The current app opens as Agent Workspaces. The online guide is documentation, not a cloud version of the app. Examples are instructional, not completed live runs.

Contents

Click a chapter to jump to it. Within each chapter, use the linked subsection list. Open your PDF viewer's bookmarks for the complete chapter/subsection outline.

Start here: what this product does	3
Understand the words and screens	4
Install and open the application	6
Connect the AI model	7
Create your first agent	9
Give an agent its first task	11
Files and human requests	13
Browser setup, sign-in, and takeover	14
Understand the code and command tools	16
Share work and coordinate agents	18
Google accounts: Gmail, Drive, and Sheets	19
Review results and repeat useful work	21
Understand Fleets before starting one	23
Data, privacy, pause, and backup	25
New controls in version 0.10.0	26
Troubleshooting and realistic expectations	28
Example 1 - Set up a competitor research agent	30
Example 2 - Set up an agent to compare two CSV files	33
Example 3 - Set up a read-only Gmail brief agent	37
Example 4 - Set up an agent to review supplied code	40
Example 5 - Set up a website usability review agent	43
Example 6 - Set up a research agent for a decision	46
Example 7 - Establish a fleet to review published files	49
Example 8 - Establish a fleet in the local website lab	54
Further reading and developer commands	57

Examples start with creating a helper and continue through inputs, limits, running, waiting, and reviewing the exact output. No prior project knowledge is assumed.

CHAPTER 01 / START HERE

Start here: what this product does

A plain-language introduction before you touch any settings.

IN THIS CHAPTER

[A simple example](#) / [What the application name means](#) / [Desktop application versus this guide website](#) / [How to read this guide](#)

Taskivra AI is a desktop application for your Mac. You give an AI helper a piece of work, give it the information and access it needs, and review what it produces. You can have several helpers with different jobs. The application saves their tasks and progress so you can supervise the work.

An agent is an AI helper with a name, instructions, private files, and a browser session. It uses a model - the AI service that reads context and proposes the next step. The application checks those proposed actions before using its tools. A task is one specific job given to an agent.

A simple example

You create an agent called Research Assistant. You give it two company website links and ask it to compare their pricing. It reads permitted pages, writes a report, and asks you if it needs missing information. You review the report and decide whether it is useful. It does not become correct merely because the task says Complete.

What the application name means

This repository is called Taskivra-Ai and this guide calls the product Taskivra AI. The current desktop window and local application still say Agent Workspaces. They are the same project. Storage and internal names have been kept so existing work is not disrupted.

Desktop application versus this guide website

The Vercel website hosts this guide and its PDF. It is documentation, not a cloud version of the application. Your agents, database, files, browser sessions, and Docker jobs run through the local Mac application. Hosted AI providers process the context sent to their APIs.

Current stage

Version 0.10.0 is a local pre-release. Apple Silicon Mac is the validated development platform. A public installer, Windows/Linux support, Intel Mac support, automatic updates, and clean-machine installation have not been validated.

How to read this guide

- If the app is not installed, start with Install and open the application, then Connect the AI model.

- If someone has already prepared your Mac, start with Create your first agent, then Your first task.
 - Choose a use-case walkthrough to see exactly what to type and how to review its output.
 - Use the clickable contents, subsection links, and PDF bookmarks to move around. Every ordinary PDF page has a Contents link.
- Related chapters: [Create your first agent](#), [Your first task](#), and [Understand Fleets](#).

CHAPTER 02 / START HERE

Understand the words and screens

Know what you are looking at and what each part is for.

IN THIS CHAPTER

[Where to go in the app](#)

Word	Plain-language meaning
Agent	A named helper with reusable instructions, its own private workspace, and a separate browser session.
Task	One job with an objective, completion criteria, allowed access, model, and limits.
Objective	The outcome you want. Example: compare two sales files and explain the change.
Completion criteria	What a usable finished result should contain. This is your definition of done.
Project	An area of work such as one client, business, or personal topic. Sharing stays within that project.
Private workspace	Managed task files and outputs available only through authorized access. It is not unrestricted access to your Mac.
Shared library	Files deliberately published to the project so authorized agents can use them. Shared does not mean public on the internet.
Version	An exact saved copy of a file. A task pins that copy so later changes cannot silently alter its evidence.
Workflow	A reusable starter that asks for the information needed for a common job.
Fleet	A bounded team organized around one objective, with a lead and specialists.
Model / provider	The AI model is the reasoning service; the provider is the service or local server that makes it available.

Word	Plain-language meaning
API key	A credential that lets the application call a provider. Enter it only in the model connection form.
Tool	An action the app permits, such as observing a page, reading a selected file, or running isolated code.
Receipt / evidence	A saved record of an observation, source, or exact output identity. It helps you check how a report was produced.
Token	A unit of model input/output text. Token limits bound request size and help estimate provider use.
Checkpoint	Saved progress used when reviewing and resuming interrupted work.

Where to go in the app

Screen	Use it for
Projects	Create an area of work, add its agents, and keep the brief and sharing scoped.
Workflows	Choose a starter such as Compare competitors or Review unread Gmail.
Agent / task view	Open a job, inspect Activity and files, run it, pause it, or respond to a request.
Requests	See what an agent needs from you: files, clarification, or a connection.
Shared library / Shared board / Agent inboxes	Find published project files, dependencies, and scoped coordination.
Fleets	Prepare one objective for a team and inspect roles, claims, messages, and results.
Results	Review the exact output, checks, acceptance state, and export options.
Routines	Set up eligible recurring read-only jobs and optional content-change alerts.
Automatic recovery	Inspect recorded transient-read incidents and their bounded retry history.
Settings	Configure model connections, Google accounts, browser setup, and backup/recovery.

A useful distinction

Agent instructions describe how a helper should generally work. A task objective describes what it should do this time. Neither is a password or a replacement for the app's permission controls.

CHAPTER 03 / SETUP

Install and open the application

Prepare a source checkout on a Mac, or open an app that has already been prepared.

IN THIS CHAPTER

[If you already have the app](#) / [If you are installing from this repository](#)

If you already have the app

01 Open Agent Workspaces

Open the local application your developer prepared. It currently uses the older product name. Read any setup status shown in Settings. A window opening does not mean its model, browser, or container tools are ready.

02 Check the model connection

Before live work, open Settings and check Models and providers. Continue with the next chapter if no usable connection is configured.

If you are installing from this repository

The repository is source code, not an automatically configured installer. These steps are for the person preparing the Mac. Terminal is the Mac application used to enter the commands below. A beginner can ask a developer to complete this section, then use the interface for ordinary tasks.

- Use an Apple Silicon Mac, with Node.js 24.14.0 or newer and npm installed. Node/npm build and install the source dependencies.
- Apple Command Line Tools provide the compiler for the native helpers. Install them with `xcode-select --install` if missing.
- Install Google Chrome if ordinary browser tasks will use the dedicated Chrome backend.
- Install and start Docker Desktop only if code execution or the optional container browser is needed. The local lab browser itself does not need Docker.

01 Download the source

Open Terminal in a suitable folder, then run the download and install commands below. `npm ci` installs the versions recorded by the repository lockfile.

02 Compile native helpers

Run the helper commands. They compile the Mac components used for Keychain and the Chrome bridge. They do not sign in, read saved secrets, or connect an account.

03 Start the app

Run `npm start` from the repository folder. It builds the interface and opens Electron, the desktop runtime.

04 Complete capability setup

Connect a model. Prepare the selected agent's Chrome profile if needed. Provision the code image if a workflow requires calculations. Each capability has a separate chapter.

```
git clone https://github.com/Ritesh102000/Taskivra-Ai.git
cd Taskivra-Ai
npm ci
npm run model:setup
npm run gmail:setup
node packages/google-workspace/setup.mjs
npm run browser:native:setup
npm start
```

Disk space and installation limits

Code-image setup checks for at least 8 GiB free host disk; local app packaging checks for at least 2 GiB. These are separate minimums, not the total space needed. Docker/browser images can use more. Clean-Mac installation and a public signed/notarized distribution are still release work.

If npm or a helper is missing, resolve that setup issue before starting a task. Do not paste API keys or your Mac password into Terminal commands. See [Troubleshooting](#).

CHAPTER 04 / SETUP

Connect the AI model

Give the app a supported AI service or a local model server.

IN THIS CHAPTER

[Choose cloud or local](#) / [Add a connection step by step](#) / [Understand task limits](#)

Choose cloud or local

A cloud connection sends task context to a provider over the network and may charge for use. A local connection points to a model server you already run on the Mac. The application does not download models or install Ollama/LM Studio for you.

Choice	What you need
OpenAI or Claude / Anthropic	An API account, exact supported model ID, API key, and current declared prices. A consumer chat subscription is not API access.
Ollama on this Mac	A running Ollama server and installed model with its explicit tag. Choose a model/server that supports the app's structured tool calls.
LM Studio or another compatible server	A running compatible server, its API base URL, exact model ID, and any authentication it requires.

Choice	What you need
OpenRouter, Together AI, Groq, LiteLLM, or another gateway	A compatible endpoint, credentials if required, an exact model, and declared metered prices when cloud charges apply.

Add a connection step by step

01 Open Settings

Find Models and providers, then choose Add connection. Pause running tasks before changing model connections.

02 Choose Provider or server

Use a preset that matches the service. The preset fills some protocol settings; it does not install or test the model.

03 Enter Connection name and Exact model ID

Give the connection a name you recognize. Copy the exact model identifier from your provider/server. Do not use a guessed model name.

04 Check API base URL, Authentication, and Billing

The base URL is the address of the model service. Remote services require HTTPS. Local HTTP is supported only for loopback. A local gateway that calls paid cloud models should still be metered.

05 Enter a key and declare prices if needed

Use the password-style API key field. Metered connections need input/output prices per million tokens. Check current prices with your provider rather than treating an example as a live quote.

06 Review per-call token limits

Reserved input tokens per call is the allowance reserved before a request. Maximum output tokens per call limits the answer. Keep both within the model's capacity.

07 Confirm structured tool support and save

Confirm that the chosen model supports structured tool calls, then select Save connection. The app says no model request was sent.

08 Select this model for the task

When preparing a task or fleet, choose the saved connection. A small harmless first task can reveal account/protocol problems; it is live use and may cost money.

Understand task limits

Limit	Why it matters
Model spend	Maximum declared model spend allowed for the task. Settled usage plus uncertain reservations count.
Model calls	How many model generation attempts the task may use.
Tool steps	How many tool actions it may perform.
Total tokens	The task's input/output token budget.
Active seconds	Time spent actively working. Waiting for human input is distinct.

Configured is not tested

Supported protocols are OpenAI Responses, Anthropic Messages, native Ollama chat, and compatible Chat Completions. The current contract supports text and at most one valid client tool call per turn. Some model dialects are incompatible. Saved settings do not prove a working account/server. Editing a connection creates a new revision; saved tasks retain the previous revision.

Keys are stored in macOS Keychain. Unknown provider outcomes retain a budget reservation and are not blindly retried. Declared-price accounting is local; invoices and gateway fees can differ. See [Data and recovery](#).

CHAPTER 05 / EVERYDAY USE

Create your first agent

Give a helper a name, reusable instructions, and the right project.

IN THIS CHAPTER

[Create a project first when you need separation](#) / [Create an agent from the main agent list](#) / [A copyable instruction example](#) / [What is separate for each agent](#)

Create a project first when you need separation

01 Open Projects

A project keeps a client or topic's shared files, task membership, and coordination separate. You can use the existing personal workspace for a first simple task.

02 Create project

In New project, enter a Name such as Practice workspace and an optional Description. Choose Create project.

03 Add an agent inside that project

Use its Add an agent form to create the helper in the correct project. Existing agents cannot currently be moved between projects.

Create an agent from the main agent list

01 Choose the plus beside YOUR AGENTS

This opens Create an agent. For client-specific work, use the project's Add an agent form instead so membership is explicit.

02 Enter Agent name

Use a job-oriented name such as Research Assistant, Data Analyst, or Mail Assistant. The name does not grant tools or account access.

03 Enter Instructions

Describe its normal working style and constraints. This is reusable guidance, not the particular task. Instructions are optional, but clear ones make the purpose easier to review.

04 Choose Create agent

The helper appears in the agent list with a private workspace. Creating it does not launch a paid model task or sign in a browser.

A copyable instruction example

```
Work carefully and explain your sources.  
Ask me when essential information is missing.  
Distinguish observed facts from assumptions.  
Do not claim to have read content you could not access.  
Write a clear result with limitations and next steps.
```

What is separate for each agent

- Private task files and outputs are scoped rather than shared automatically.
- Browser cookies and tabs belong to the agent session/profile, not your personal browser.
- Tasks have their own objectives, model selections, access, and limits.
- Publishing a file is an explicit way to let approved project peers use an exact copy.

Keep secrets out of instructions

Do not put your API key, Google password, Mac password, or personal browser cookies here. Use Settings and direct sign-in when a connection or login is needed.

Next: [give the helper its first task](#). Use-case chapters later provide a suitable name and instructions for each example.

CHAPTER 06 / EVERYDAY USE

Give an agent its first task

Turn a vague request into a saved job you can inspect and run.

IN THIS CHAPTER

[The easier route: a workflow](#) / [The manual route: New task](#) / [How to write a useful objective](#) / [How to define done](#) / [Understand progress states](#)

The easier route: a workflow

01 Open Workflows

Select a starter that matches the job, such as Compare competitors, Compare two CSV files, Review code files, or Review unread Gmail.

02 Choose Use workflow

Read the requested fields and enter the job-specific information. A workflow is a form plus a saved procedure, not a guarantee of a good result.

03 Choose the project agent and model

Select the helper that should do the job and its configured model. Review the permissions and budget controls shown.

04 Review task, then Save task

Review the prepared objective, criteria, and allowed access. Saving prepares the job; it does not mean the model has completed it.

05 Resolve readiness

Open the saved task. Assign required files or configure the missing browser, model, or account. Readiness reports what is missing.

06 Choose Run agent

This starts live model calls and permitted tools. Keep the app available. Inspect Activity as it works.

07 Respond when it needs you

Open the request, answer only the missing information, or complete browser/account setup. Paused or cancelled work stays under your control.

08 Review Results

Open the exact output and check its sources and coverage. Accept it if useful, or request changes with a precise correction.

The manual route: New task

Use New task on the selected agent when no starter fits. Choose Live agent for real model/tool use, enter What should the agent accomplish? and What does done look like?, select the model, and review access/limits. Save live task, then resolve readiness and run. Live mode is different from Simulation.

How to write a useful objective

```
Compare the public pricing pages I approved for Company A
and Company B. Summarize the plans, important differences,
and any information you could not verify.
```

How to define done

```
A private Markdown report with a comparison table,
links to the pages actually read, uncertainties,
and three questions I should investigate next.
```

Understand progress states

State	What it means
Draft / prepared / ready to run	Saved configuration exists; live work has not necessarily started.
Queued / working	The job is waiting for capacity or actively using its configured model/tools.
Needs input / waiting	An agent needs a file, answer, or connection. This is not a completed task.
Paused	Execution is stopped temporarily. Review saved progress before resuming.
Complete	The application accepted the completion path and saved an output. You still need to review its accuracy.
Failed / stopped	The job cannot continue or was ended. Historical progress and errors remain available.

Try simulation if you only want to learn the controls
Simulation uses scripted local scenarios such as asking clarification, completing, or failing. It makes no model calls. It does not demonstrate that a live AI has performed the same work.

For complete examples, choose a [research walkthrough](#) or the [CSV walkthrough](#).

CHAPTER 07 / EVERYDAY USE

Files and human requests

Give the right inputs without sharing your whole computer.

IN THIS CHAPTER

[Import a private file](#) / [When the agent requests a file](#) / [Clarification and reduced scope](#) / [Publish only when another agent needs the file](#)

Import a private file

01 Open the intended task

Choose the job that should use the file. Importing into an unrelated task does not grant this job access.

02 Use the file picker or supported file drop

Select only the files needed. The app imports managed copies and validates format/size. Agents do not browse arbitrary host folders.

03 Check the saved version

A version is the exact imported copy. Later edits to the original file on your Mac do not silently change it.

04 Assign named workflow slots if present

In Activity, open Ready for this job? and Assign the files for this run. Give each required slot the correct version, such as the earlier and later CSV. Read the validation result.

When the agent requests a file

01 Read the request and slot names

A slot explains the particular file or format needed. Do not assume any file satisfies it.

02 Supply the requested input

Choose a matching file through the owner interface. Do not paste a local file path as a substitute for importing the content.

03 Resolve rejected slots

A wrong format or missing slot remains blocked. Replace the rejected input; already accepted slots remain saved.

04 Review continuation

Once all required inputs are fulfilled, the eligible task can receive one continuation from saved progress. An owner pause or cancellation is not overridden.

Clarification and reduced scope

A clarification request asks a question such as which date range to analyze. Answer with the specific missing information. If the agent cannot complete the original scope, it can ask you to accept a smaller scope. The application requires your acceptance rather than silently calling incomplete work done.

Publish only when another agent needs the file

A private output stays private until deliberately published. Publish an exact project version, then authorize a peer task to consume it. The shared library is a project sharing area, not a public website. If you publish a later copy, tasks already using the earlier version keep their original evidence.

Bounds and supported formats

Default import bounds include 100 MiB per file, 250 MiB per batch, and 32 files per batch. A workflow, preview, Google import, or fleet can impose smaller limits. PDF/XLSX extraction uses a separately prepared document image and supports ordinary bounded files; it does not run macros or perform OCR.

Related: [sharing and collaboration](#), [isolated calculations](#).

CHAPTER 08 / SETUP

Browser setup, sign-in, and takeover

Let an agent browse separately while you keep using your own browser.

IN THIS CHAPTER

[Choose the right browser path](#) / [Prepare dedicated Chrome](#) / [Take control for login](#) / [Can you work while the agent works?](#)

Choose the right browser path

Path	Use and limits
Dedicated Chrome	Ordinary agent websites, a separate local profile per agent, and direct owner login. It runs on the Mac and uses normal host networking.
Optional container browser	Docker-isolated browser with managed file-transfer support and optional encrypted login checkpoints. Fresh-machine base-image setup is not complete.
Local lab browser	Temporary independent Electron sessions for the bundled fictional website only. No Chrome extension is needed; closing clears login storage.

Prepare dedicated Chrome

01 Install Chrome and compile the bridge

Chrome is expected at /Applications/Google Chrome.app. The source helper is built with `npm run browser:native:setup`.

02 Open the selected agent's Browser setup

Prepare its dedicated profile, then select Open Chrome setup. Use that agent's Chrome window, not your personal profile.

03 Open chrome://extensions

Enable Developer mode, choose Load unpacked, and select extensions/agent-browser inside this checkout.

04 Return to the app and refresh status

The bridge connects that particular agent profile. Each new profile needs its own extension setup.

05 Reload after changing extension source

Close the task browser, Reload the extension in its Extensions page, then reopen the browser. Rebuilding the app alone does not reload an unpacked extension.

Take control for login

01 Open the agent browser

Inspect the task's browser state. If the task requests login, read which account/site it needs.

02 Choose Take control

The app transfers exclusive control to you. Use the actual dedicated Chrome window for navigation, password entry, and MFA.

03 Finish the intended login

Credentials stay out of task messages and saved page traces. Do not import your personal Chrome profile or enable personal profile sync.

04 Choose Return to agent

The app adopts allowed relevant tabs and requires a fresh observation before agent actions resume. The task still respects its allowed sites and saved scope.

Can you work while the agent works?

Ordinary agent operations use their managed tabs rather than the Mac's global mouse/keyboard. You can use your own browser while agents work in separate sessions. Explicit inspection/takeover can bring the agent window forward. Human takeover blocks agent reads, previews, and inputs until you return control.

Login compatibility is not guaranteed

A dedicated Chrome profile separates cookies from your personal browser, but it is not a container/network sandbox. A site can still reject login or ask additional checks. Gmail has a supported read-only OAuth alternative. Native managed upload/download is unavailable; manual downloads do not automatically become task inputs.

There are two active browser slots and up to six managed tabs per session. Backends do not migrate cookies when switched. For Google accounts see [Google connections](#).

CHAPTER 09 / SETUP

Understand the code and command tools

Calculations happen in isolated jobs, not an unrestricted terminal on your Mac.

IN THIS CHAPTER

[Prepare the code runtime](#) / [What the agent can and cannot reach](#) / [The lab command tool is different](#)

Some tasks need calculations or file transformation. Python and Node.js are programming runtimes; shell is a command language. The app can execute these inside a disposable Docker container. A container is a separate execution environment with bounded resources.

Prepare the code runtime

01 Install and start Docker Desktop

Docker must be running before the app can prepare or launch jobs. The app does not install it automatically.

02 Explicitly provision the portable image

On a fresh Apple Silicon Mac, run the command below. Network access is allowed during this owner-controlled image build, not during agent payload jobs.

03 Check readiness

Run `npm run code:check` and resolve the reported issue if not ready.

04 Run only an authorized task

The task receives verified snapshots of approved input versions. It can create bounded work/output files; exported files are verified before commitment.

```
node packages/code-runtime/setup.mjs --build \  
  --bootstrap-base --allow-network  
npm run code:check
```

What the agent can and cannot reach

- Jobs run offline with no task-time network. They do not receive host folders, browser cookies, model keys, or the Docker socket.
- Standard images have Python/Node standard libraries. Additional packages require a reviewed pinned dependency recipe.
- Default job limits include one CPU, 1 GiB memory, a 120-second payload deadline, and bounded logs/output.
- Stop interrupts the owned container and fences late output commitment. A clean exit alone is insufficient; export must also verify.

The lab command tool is different

In the local website fleet, `lab_command` accepts a small curl-shaped JSON request to the same verified training site. It performs one bounded HTTP request using that agent's synthetic session. It does not start a host shell, execute the Mac's curl, read host files, or contact arbitrary targets. Offline code in that mode can process permitted fleet reports only.

Fresh setup distinction

`npm run code:setup` is the older offline build path and expects the exact retained Phase 0 base. A missing base on a new machine requires the explicit portable provisioning command above. The optional container browser has a separate setup gap.

CHAPTER 10 / TEAMWORK

Share work and coordinate agents

Understand what becomes visible and how handoffs stay reproducible.

IN THIS CHAPTER

[Private first, shared deliberately](#) / [Ordinary coordination versus Fleets](#)

Private first, shared deliberately

Agents do not automatically see each other's private files, objectives, browser pages, or unpublished outputs. A project gives them a shared library, task awareness, and scoped inboxes. Publishing and task authorization decide what evidence another agent can use.

01 Produce or import the source

Keep it in the relevant project/task. Check that it contains only what the peer needs.

02 Publish its exact version

This creates a shared immutable project copy. Publish does not upload it to the public internet.

03 Select that version for the receiving job

The peer works from the pinned copy. Describe what it should do with it and define a useful output.

04 Use messages for coordination

Send scoped status/findings through the permitted coordination surface. General messages primarily share references and summaries; they do not automatically grant private file or browser access.

05 Review dependencies

A dependent task needs the upstream work to complete. Cycles and unfinished/failed upstream dependencies remain visible rather than being treated as satisfied.

06 Review the handed-over output

Record exact report versions and source coverage. Do not assume a peer message proves a conclusion.

Ordinary coordination versus Fleets

Ordinary agents can coordinate within authorized peers/projects. Fleets add an automatically organized bounded lead-and-specialist workflow. A fleet lead plans items and dependencies, workers exclusively claim eligible work, scoped messages carry bounded finding text, and exact completed reports are staged for the lead or dependent workers.

Version pinning matters

If a report changes tomorrow, yesterday's task does not silently consume the new report. A new input selection or handoff records the intended exact copy. This makes it easier to reproduce what each agent actually saw.

For a single objective handled by a team, continue with [Understand Fleets](#).

CHAPTER 11 / SETUP

Google accounts: Gmail, Drive, and Sheets

Connect explicitly and understand exactly what is read.

IN THIS CHAPTER

[Why Gmail uses a separate connection](#) / [Connect Gmail step by step](#) / [Basic and detailed mail reads](#) / [Selected Drive and Sheets imports](#)

Why Gmail uses a separate connection

Google can reject sign-in inside an embedded automated browser. The supported Gmail path uses a Desktop OAuth connection: you sign in and consent in the system browser, then the app reads through a bounded API. OAuth means granting a specific application permission without placing your Google password in a task.

Connect Gmail step by step

01 Prepare your Google Cloud project

In Google Cloud Console, create/select the project that will own this connection. Enable Gmail API.

02 Configure the consent audience

Set up app branding/contact details and the appropriate audience. If it is in Testing, list the exact intended Gmail address as a test user.

03 Choose the read-only scope

Configure gmail.readonly. The app has no Gmail send, archive, delete, modify, or mark-read operation.

04 Create a Desktop app client

Create an OAuth client with application type Desktop app and download its JSON. A Web client or service-account key is not the required format.

05 Prepare and save the Gmail task

Open Workflows > Review unread Gmail > Use workflow. Select the intended agent, enter your exact Gmail address and what matters to you, choose a configured model, and review the limits. Choose Review task, leave detailed review disabled for a headers-and-snippets brief, then choose Save task. A saved live task naming the account is required before Connect Gmail becomes available. Saving does not read mail or start model calls.

06 Import and connect in Settings

In Settings > Gmail connection, import the JSON through the native picker. If several saved mail tasks exist, select the intended Saved Gmail task. Choose Connect Gmail and complete sign-in/consent in the system browser with the exact account saved on that task.

07 Verify the account and approve project use

Choose Verify account for project use, then open Projects and select the project containing the agent. Under Approved accounts > Gmail, choose Approve followed by the verified address. The exact account must match the saved task/project requirement; a different account cannot release that blocker. After restart, Verify account checks identity without listing mail. Return to the saved task and check its readiness before choosing Run agent.

Basic and detailed mail reads

Mode	What it reads
Unread brief	Up to 50 unread message headers/snippets with pagination and truncation disclosed. Reading preserves unread status.
Explicit detailed task	Bounded search, selected plain-text thread reads, and supported selected attachment imports. Full-body tools are not added to older tasks automatically.
What neither mode does	Send, archive, delete, mark read, fetch remote images, or execute active email HTML.

Selected Drive and Sheets imports

Drive/Sheets use a separate connection and separate project account approval. Enable the relevant APIs, import a Desktop client through the Google-inputs flow, and choose one exact supported file or finite named-sheet A1 range. The app imports an immutable local snapshot with source identity, retrieval time, hash, and omissions. It does not modify or continually synchronize the source.

Account permission versus selected resource

The current Drive read-only OAuth scope is broad. The application broker restricts which selected resource it imports; it is not a Google-issued per-file grant. Gmail tokens do not grant Drive authority. The original downloaded client JSON remains where you saved it, so keep it private.

Expired or revoked authorization requires reconnection. Provider setup/verification for a widely distributed app is separate release work. See the [email walkthrough](#) for a copyable first task.

CHAPTER 12 / EVERYDAY USE

Review results and repeat useful work

Completion, acceptance, export, and scheduling mean different things.

IN THIS CHAPTER

[Review the result](#) / [Choose an export](#) / [Save a reusable workflow](#) / [Set up a routine carefully](#)

Review the result

01 Open Results or the completed task

Read the exact output version, rather than only the progress summary.

02 Read coverage and evidence

Check which pages/files/messages were read and what was omitted. Confirm important assertions against their sources.

03 Inspect quality checks

They can catch malformed JSON/CSV, missing required headings, empty output, or receipt references outside the task. Passing does not prove facts or calculations.

04 Accept or request changes

Accept records your judgment. If something is wrong, ask for a specific correction and retain the original version history. Fleet result revisions follow their own scoped workflow.

05 Export if needed

Use the available exact-original export or a supported formatted report export. The native save dialog selects a destination and refuses replacement of an existing file.

Choose an export

Output	Supported formatted export
Markdown / plain text	PDF or Word DOCX with source/version identity. External resources and scripts are not loaded.
CSV	Excel XLSX with literal text cells, filters, and a source-record sheet; formulas are not executed.
Unsupported or oversized content	Exact original export. A formatted export that cannot fully represent the content is rejected rather than silently clipped.

Save a reusable workflow

Save a suitable live task as a workflow when you want the same reviewed brief again. A reusable procedure can keep named input fields and output requirements, while each use requires fresh inputs. It does not copy private files, messages, account sessions, or existing grants. Review a brief for personal details before reusing it.

Set up a routine carefully

Routines schedule eligible accepted read-only workflows with per-run/monthly limits and no overlap. This Mac and app must be available; it is not an always-on cloud scheduler. Missed, uncertain, or interrupted occurrences are not blindly replayed. Optional in-app alerts compare a new exact output with an earlier accepted baseline.

An alert does not prove the world changed

It detects content differences, which can include different model wording. Imported files/Google snapshots do not automatically refresh. Acknowledge an alert to mark it reviewed; that does not accept its output.

See [backup and recovery](#) before moving or restoring work.

CHAPTER 13 / TEAMWORK

Understand Fleets before starting one

One objective can create a team, but tools and limits still apply.

IN THIS CHAPTER

[How a fleet works](#) / [Choose a mode](#) / [Draft versus start](#) / [Default team and task limits](#) / [Pause, stop, and restart](#)

How a fleet works

01 You set one objective

Choose the project, mode, exact allowed evidence where needed, lead/worker models, and team/task limits.

02 The lead plans

It proposes bounded specialist roles, work items, and dependencies. Creating a plan does not mean the items are completed.

03 Workers claim eligible items

A claim makes one worker responsible for an item. Independent items can work concurrently if the plan and available capacity permit it.

04 Agents exchange findings and reports

Scoped messages coordinate the team. Exact completed report versions are published/staged for the lead or dependent workers.

05 The lead revises or waits

It can add bounded follow-up work when evidence calls for it. Dependencies and limits still apply; it releases browser capacity while waiting.

06 The lead produces a combined result

Once required items finish, the lead can synthesize the reports. The owner still reviews source support and coverage.

Choose a mode

Mode	Starting inputs and available work
Review published files	Selected immutable text evidence and fleet reports. No live browser, network target, or code execution. Static concerns must be labeled as potential rather than live-verified flaws.
Test local website	The app-owned fictional Harbor Desk URL. Real separate ephemeral lab browsers, a bounded local HTTP checker, and offline code over fleet reports if its container is ready. No attached site source or hidden answer key.

Draft versus start

Save draft makes no model calls. Start fleet authorizes planning and internal handoffs within the saved models, scope, and limits. It does not grant unrestricted tools or remove the need for compatible models.

Default team and task limits

Default	Team / fleet	Each task
Agents / concurrency	4 including lead / 2 working tasks	One executing task per agent
Model spend	USD 5	USD 2
Model calls	100	30
Tokens	500,000	150,000
Active seconds	1,800 combined	600
Other bounds	8 work items; 3 plan revisions	50 tool steps

These are editable defaults inside validated maximums, not a suggested bill or guaranteed completed run. Team and task limits both apply. Settled spend and uncertain reservations count. Handoffs and new work do not reset budgets.

Pause, stop, and restart

Pause fleet preserves progress and closes lab browsers. Stop fleet is final for execution and leaves the history. Restart, sleep, or backup requires explicit review/resume. Lab cookies and website state are temporary, even when task plans and reports are durable.

What has and has not been demonstrated

Scripted integration tests demonstrate overlapping workers, claims, messages, report handoffs, and synthesis through the real application mechanics. The paid local-website assessment is paused and incomplete: no final report, verified model-discovered flaw, or simultaneous live specialist proof. Model quality and its plan determine actual outcomes.

Walkthroughs: [published evidence fleet](#) and [local website fleet](#).

CHAPTER 14 / REFERENCE

Data, privacy, pause, and backup

Know what stays on the Mac and what can leave it.

IN THIS CHAPTER

[Where work is saved](#) / [What a model receives](#) / [Create and verify a backup](#) / [What automatic recovery handles](#)

Where work is saved

The default persistent root is ~/Library/Application Support/Agent Workspaces. SQLite stores task/run history, requests, receipts, usage, projects, and fleet state. Managed directories store exact artifacts and workspace revisions. This local data is not the public source repository.

What a model receives

A hosted model receives the selected task context and authorized content needed for its requests. A local model server processes requests through the chosen endpoint. Private by default refers to app sharing boundaries; it does not mean content is never sent to a configured cloud provider.

- Model API keys and Google connection/client tokens use macOS Keychain and are excluded from ordinary task files and portable backups.
- Native Chrome manages its own dedicated profile storage; container browser checkpoints are encrypted. They are different stores.
- Files and messages you supply can contain secrets. The app does not automatically redact every possible sensitive item.
- Local portable backups contain private tasks/files and are not encrypted or content-redacted. Keep them private.

Create and verify a backup

01 Open Settings: Backup and recovery

Choose Create backup. This pauses work while the app makes a verified copy.

02 Choose a new destination in the Mac dialog

The app writes to a new child location and refuses replacement. Keep the backup somewhere private with enough space.

03 Use Check backup

Verification checks exact hashes, referenced files, database integrity, and foreign-key consistency.

04 Restore to new folder when needed

Restore creates a new verified location; it does not overwrite the active workspace or automatically switch to it. Credentials and browser logins are not transferred.

05 Review before reactivating

Restored tasks and routines stay paused/disabled until reviewed. Accounts and browser sessions require appropriate fresh setup.

What automatic recovery handles

The recovery service can retry transient browser observations within durable finite allowances. It does not blindly retry login/MFA, paid model requests, uncertain navigation, clicks, or other writes. Inspect Automatic recovery for the saved history. Acknowledge marks an incident reviewed; it does not reset its retry allowance.

Protected system prompts

Complete macOS Keychain access dialogs yourself after reviewing them. Do not send your Mac password to an agent or paste it into messages. Rebuilt apps can be treated as changed by macOS.

CHAPTER 15 / USING THE APP

New controls in version 0.10.0

Use the new source, history, access, resource and recovery controls safely.

IN THIS CHAPTER

[Capture a reviewed repository source pilot](#) / [Read source coverage and result provenance](#) / [Inspect access and the next-call resources](#) / [Read history and archive finished work](#) / [Follow fleet questions and create fresh follow-ups](#) / [Use recovery and runtime diagnostics](#) / [Choose a version and plan rollback](#)

Capture a reviewed repository source pilot**01 Choose a project agent**

Open Fleets and the Capture a repository source pilot section. Select an active agent belonging to the current project. If none exists, create one through Projects.

02 Review before saving

Choose folder and review snapshot opens the native folder picker. Read Captured files, Exclusions and limits, and Read captured content before saving. Paths are labels; the app does not execute repository code.

03 Save the exact reviewed bytes

Choose Capture reviewed snapshot. The app saves the cached immutable JSON version into this project's shared library. Select that published version as fleet evidence separately; capture does not start a model call.

A bounded pilot

Capture includes at most 16 text files, 8 KiB per file, 32 KiB total source and 64 KiB JSON, within 256 visited entries and depth 8. Symlinks, hardlinks, possible secrets, binary and oversized files are excluded. Secret detection is heuristic, so review every captured file before sharing. Unsupported packed Git objects or worktrees show unknown metadata. This feature requires `/usr/bin/python3` and does not establish full repository coverage.

Read source coverage and result provenance

Exact source ranges let a task read bounded intervals from an authorized immutable version. Selected inventory, retrieved intervals, reviewer-declared inspection and validated findings are different records. In Results, inspect exact source/output identities, named roles, execution receipts, assessment and owner acceptance. Structural checks do not establish factual truth. Same-task supporting outputs remain separately identified.

Inspect access and the next-call resources

In a task's Activity panel, inspect exact current grants. Pause the task before ending future use. Revocation does not undo earlier downloads, publications or external effects. The local resource preview explains next-call fit, remaining headroom and held reservations without contacting the provider. Unknown dispatch holds are retained; do not assume a failed display means a model call was never sent. A smaller output allowance can help when the fixed allowance alone exceeds the remaining budget.

Read history and archive finished work

Open All tasks, then History and archive. Agent, task and fleet history uses bounded pages and exact selections. Archive finished work only after operations and runtime cleanup are resolved. Archive frees active occupancy and retains historical identities and file bytes; it is not deletion. Restoring checks capacity and does not start work. Retained bytes still count toward storage.

Follow fleet questions and create fresh follow-ups

Fleet mailboxes link questions to their exact replies and support bounded older-message reads. A follow-up creates a fresh paused draft with explicit source lineage and newly reviewed limits. Review evidence and models, then start separately. It does not clone grants or silently resume a failed worker.

Use recovery and runtime diagnostics

Requests identify typed attention causes and supported next actions. Where available, repair the exact saved replan explanation within its finite admission limits. A quarantined file repair requires the owner to select bytes with the exact expected hash and size; it preserves the version identity. Code history shows effective limits, immutable workspace changes and cleanup state separately from execution outcome. Browser setup checks native extension compatibility before declaring readiness. Backups contain private content and are unencrypted.

Choose a version and plan rollback

GitHub release tags preserve each published source version. The hosted guide has a Guide versions page with matching editions and PDFs. This release migrates the database from schema 18 to schema 24. Returning to version 0.9.9 requires compatible data preserved before the upgrade; checking out old source does not downgrade the database. Close the app and runtimes before preserving data, and keep credentials and browser profiles separately protected.

Read the [version and rollback instructions](#) and the [complete improvement ledger](#).

CHAPTER 16 / REFERENCE

Troubleshooting and realistic expectations

Use the saved error and readiness state to choose the next step.

IN THIS CHAPTER

[What to include in an issue report](#) / [Current evidence](#)

If you see this	What to do
App opens but tasks cannot run	Open readiness and Settings. A saved model, native helper, browser profile, Google account, or code image may still be missing.
Model connection not tested	The configuration is saved, not proven. Check exact model/protocol, credential, usable token counts, and the app's single-tool contract. A live harmless probe may cost money.

If you see this	What to do
Chrome disconnected	Check the extension in the correct dedicated agent profile. Reload changed source and reopen the task browser; allow a reconnect interval.
Gmail login rejected	Use Desktop OAuth through the system browser. The app cannot promise to bypass a provider's login decision.
Google wrong account / expired consent	Verify the exact account/project binding and reconnect. A different account does not satisfy the original request.
Pinned code base missing	Run the explicit portable --bootstrap-base --allow-network provisioning path after checking Docker/disk space.
Container browser base missing	It has a separate unresolved fresh-machine provisioning dependency. Native Chrome may serve an ordinary browsing task.
File input remains blocked	Read the named slot validation. Replace only invalid/missing inputs and preserve accepted versions.
Task waits after a response	Check whether it remains owner-paused, cancelled, missing another input, or blocked by upstream work.
Fleet does not work concurrently	Concurrency is a ceiling. Dependent items can be serialized by the model's plan; inspect dependencies, claims, and capacity.
Fleet reaches a budget or unknown outcome	Inspect settled and reserved usage. Review history before resuming. New work must not be used to evade saved limits.
Lab cannot start on port 4318	Another service may own the port. The app refuses it rather than treating it as the verified lab or terminating it.
Formatted export unavailable	Check format and complete coverage bounds; use exact-original export for unsupported/oversized content.

What to include in an issue report

- The app version, browser/model backend, task mode, and the precise visible error.
- What you expected, what happened, and minimal steps using harmless synthetic files.
- Which setup checks were performed and which remain unverified.
- Redacted context only: no keys, OAuth JSON, mailbox contents, passwords, browser profiles, or private backups.

Current evidence

On 1 October 2026, publication checks passed type checking, the production build, and 663 Electron regression tests, with 18 optional skips and no failures. Skips are not live proof. The latest lab navigation readiness change still needs its dedicated real-browser harness rerun; the paid model assessment remains incomplete.

Judge the result, not the animation

A busy progress view, a valid tool receipt, or a passed structure check does not prove a correct conclusion. Check sources, reproduced observations, calculations, and explicit coverage before accepting important work.

CHAPTER 17 / STEP-BY-STEP EXAMPLES

Example 1 - Set up a competitor research agent

Use this when you want a clear comparison of products from public pages you choose. The example compares project-management products for a small agency; it does not contain a completed comparison.

IN THIS CHAPTER

[What you need](#) / [Create this agent](#) / [The job and definition of done](#) / [Example inputs to enter](#) / [Follow the steps](#) / [Check the finished result](#)

Before following this example

Illustrative prompts to copy and adapt - this example has not been run

What you need

- The desktop app installed and opened, plus a model connection configured in Settings > Model connections.
- Public HTTPS pages for the products you want to compare. A workflow accepts at most twenty page links, one per line.
- Chrome and the app's browser helper installed, with the extension connected in this agent's dedicated profile.

Create this agent

Agent name: Competitor research assistant

Copy or adapt these reusable agent instructions:

Write clear comparisons for small businesses. Separate facts you observed from your interpretation. Cite the exact pages you read, record missing information, and ask for clarification when the approved sources cannot support a conclusion. Keep your working files and reports private.

The job and definition of done

Example objective:

Compare Trello and Asana for a five-person agency. Focus on published pricing, limits of the entry-level plans, client collaboration, and how easy the visible onboarding information is to understand. Explain important differences and list questions I should verify before choosing. Use only the approved pages, and do not create accounts or submit forms.

What a useful completed result should contain:

One private report.md with a comparison table, practical differences for a five-person agency, exact sources and pages actually inspected, missing information, and limits of the review.

Example inputs to enter

Use the fields named in the steps below. These are example values to adapt, not an automatically configured or completed task.

Focus / review question:

Compare Trello and Asana for a five-person agency. Focus on published pricing, entry-level plan limits, client collaboration, and clarity of the visible onboarding information. Separate observed facts from your interpretation. List questions to verify before choosing. Do not create accounts or submit forms.

Approved pages, one per line:

<https://trello.com/pricing>
<https://asana.com/pricing>

Follow the steps

01 Create the agent and give it lasting guidance

Choose the + beside YOUR AGENTS. In Create an agent, enter Competitor research assistant in Agent name and copy the example agent instructions into Instructions. Choose Create agent. This gives the assistant its own task history, private files and browser profile. Instructions describe how it should work across tasks; they do not start work or grant website access.

02 Choose the guided workflow

Open Workflows, find Compare competitors, and choose Use workflow. The workflow is a prepared task form that supplies a sensible report structure and read-only website access. In Agent, select the agent you just created. The alternative Create a new agent... creates an agent with a name but no custom instructions, so the earlier creation step is useful here.

03 Enter the question and the sources

Copy the example focus value into What are you comparing? Copy the example websites value into Pages the agent may visit, with one complete HTTPS URL per line. Replace the example products or links if your decision is different. The listed website addresses establish the allowed websites; mentioning another website in the question does not grant access to it.

04 Choose the model and bound the run

In Model, choose a connection you have already configured. Set Maximum model spend (USD) to an amount you are willing to allow, such as 1 for a first attempt. Open Advanced run limits and review Model calls, Tool steps, Active seconds and Total model tokens. You can keep their starting values for a small trial. A limit can end the attempt before the report is complete; these numbers are ceilings, not a promise about cost or completion.

05 Review, then save the task

Choose Review task. Read Your task brief, What done looks like and Access for this task. Confirm the websites and read-only behavior match your intention, then choose Save task. This saves a separate task without starting model calls. The workflow supplies its own objective and completion criteria; the example objective above explains the intended outcome, rather than being another field you must find.

06 Finish browser setup for this agent

Open the saved task's Browser panel and expand Browser setup. Use Desktop Chrome · dedicated profile. Choose Open Chrome setup when one-time setup is needed. In that Chrome profile, enable Developer mode at chrome://extensions, choose Load unpacked, and select the extension folder shown by the app. Return and use Check extension. Each agent profile needs its own setup. You can then open Activity and check Ready for this job?.

07 Start and respond when necessary

Choose Run agent on the saved task when setup is ready. The selected model now begins live work within the saved scope and limits. Watch the task conversation and Activity. If a page is unavailable or cannot answer the question, answer the request in its card. For a required sign-in, use Take control in Chrome and then Return to agent; enter credentials only on the website. This example normally uses public pages.

08 Review the report before relying on it

When the task completes, open Results or choose Review result in Activity. Read the comparison and its sources. Use Accept this version when it meets your needs, or Request changes to prepare a separate revision task. A revision remains paused until you start it. PDF report and Word document can export supported complete text reports; Export exact file preserves the original saved file.

Check the finished result

- Do the plan prices, currencies, billing periods and team-size assumptions match the actual source pages?
- Does the report cite pages that were inspected and disclose any pages it could not reach?
- Are judgments about usability presented as judgments, rather than as completed user tests?
- Are missing facts and questions to verify visible before you make a purchasing decision?

Scope and limitations

The task reads approved websites and writes private outputs. It does not sign up for products or prove that every marketing claim is correct. Read-only browsing cannot perform a full interactive onboarding test. A redirect to an unapproved website may be blocked; if you want that source included, review its address and prepare a task with the correct approved links. Result checks do not certify factual accuracy.

CHAPTER 18 / STEP-BY-STEP EXAMPLES

Example 2 - Set up an agent to compare two CSV files

Use this when you have two exported tables and want a reproducible report about what changed. A CSV is a plain-text table commonly exported from a spreadsheet. You supply the files; the app does not collect sales data for you.

IN THIS CHAPTER

[What you need](#) / [Create this agent](#) / [The job and definition of done](#) / [Example inputs to enter](#) / [Follow the steps](#) / [Check the finished result](#)

Before following this example

Illustrative prompts to copy and adapt - no example CSVs are imported automatically

What you need

- A configured model connection.
- Two CSV files: one for the current period and one for the comparison period. For this example, each should contain product, orders and revenue columns, with the same currency and compatible meanings.
- Each assigned CSV must contain at least one data row, no more than 10,000 rows and no more than 1 MiB.
- Docker Desktop running and the app's isolated code runtime prepared. The code job reads approved file copies and runs offline.

Create this agent

Agent name: Sales comparison assistant

Copy or adapt these reusable agent instructions:

```
Explain numerical findings in plain language. Check file roles, column meanings, currencies and missing values before calculating. Use reproducible calculations and show the checks you actually ran. Never invent missing data. Keep supplied data and outputs private.
```

The job and definition of done

Example objective:

```
Compare product sales between the two assigned periods. Calculate total orders and revenue in each period, absolute changes by product, and percentage changes where the comparison value is nonzero. Identify the products contributing most to the revenue change. Explain missing products, blank values, duplicate rows and any assumptions.
```

What a useful completed result should contain:

One private report.md with Findings, Coverage, Calculation checks and Limitations, identifying the exact two inputs and rows analyzed, plus a reproducible calculation appendix.

Example inputs to enter

Use the fields named in the steps below. These are example values to adapt, not an automatically configured or completed task.

Analysis question:

```
How did product sales change between these periods? Compare total orders and revenue, absolute and percentage changes by product, and products contributing most to the revenue change. Use the product, orders and revenue columns. The revenue values use the same currency. Flag duplicate products, missing values and zero comparison values before drawing conclusions.
```

What the files represent:

```
Current-period sales and comparison-period sales CSVs. Both use product, orders and revenue columns, the same currency, and one row per product.
```

Follow the steps

01 Create a private analysis agent

Choose the + beside YOUR AGENTS. Enter Sales comparison assistant in Agent name and copy the example agent instructions into Instructions, then choose Create agent. This agent will keep the imported data private. Its general instructions describe careful analysis; this particular comparison is defined in the next task.

02 Choose the two-file workflow

Open Workflows > Compare two CSV files > Use workflow. Select Sales comparison assistant in Agent. Copy the example question value into What should the analysis answer? and the example files value into What do the two periods represent? Adjust those descriptions to match your real files. The workflow expects two CSVs; it does not accept a spreadsheet workbook in place of these named slots.

03 Select a model and review limits

Choose your configured connection in Model. Review Maximum model spend (USD) and Advanced run limits. For a first small dataset, you can begin with the starting values and a spend ceiling of 1 USD if that fits your budget. Code-runtime resource limits are separate from model-call limits. Keeping a small spend ceiling may produce a partial attempt rather than a finished analysis.

04 Review and save without running

Choose Review task. Confirm there are no website addresses granted and that the task is intended to keep inputs and results private. Read the required result and choose Save task. Saving creates the task; it does not import files or start model work. Files to assign after saving lists Current-period CSV and Comparison-period CSV.

05 Import both files into this saved task

Open the task's Files panel and choose Import files. Select the two CSV files. Importing makes managed copies and attaches the exact saved versions to this task; it does not give the agent general access to folders on your Mac. Check that both appear under Inputs. If an import fails validation, correct the file and import its replacement.

06 Assign the roles explicitly

Open Activity. In Ready for this job?, find Assign the files for this run. For Current-period CSV, choose the file containing the current period. For Comparison-period CSV, choose the earlier or other period. Wait for each slot to say Checked. A slot is a named place for a required input. This assignment prevents the agent from guessing roles from similar filenames; importing alone is not enough.

07 Check the calculation runtime

In Activity, resolve any isolated-code setup item and use Check again. Docker and the prepared runtime must be available before this workflow can run its calculations. If setup is missing, use the guide's code-runtime setup instructions. Passing CSV shape checks means the files are structurally valid; it does not establish that their business meanings are comparable.

08 Run, then answer data questions

Choose Run agent. If the agent finds different currencies, missing columns or ambiguous duplicates, answer the clarification request in the task card. A proposed smaller analysis needs your acceptance; do not treat a missing required file as successfully supplied. Pause or Stop remains available if you need to halt the work.

09 Inspect the calculation record and decide

Open Results after completion. Read Findings, Coverage, Calculation checks and Limitations, including the calculation appendix. Verify that the correct periods were used and that the checks were executed. Choose Accept this version or Request changes. Export the report with PDF report or Word document when supported, or Export exact file for its original form.

Check the finished result

- Are the current and comparison periods assigned correctly, with their exact input versions named?
- Do the analyzed row counts and columns match the supplied tables?
- Were totals, duplicate handling, missing values, currencies and zero denominators checked?
- Does the report include executed calculation checks and reproducible code, rather than merely suggesting checks?
- Are business explanations separated from changes that the data alone can establish?

Scope and limitations

This is a bounded offline calculation over two supplied CSV snapshots. The workflow cannot read a live shop or spreadsheet service automatically. The starting code image supplies standard libraries; unavailable packages require a reviewed dependency request. Structural result checks do not independently confirm every calculation or the meaning of your source data.

CHAPTER 19 / STEP-BY-STEP EXAMPLES

Example 3 - Set up a read-only Gmail brief agent

Use this when you want a short list of unread messages worth your attention. This example deliberately uses headers and snippets only: a snippet is the short preview supplied by Gmail, not the full message.

IN THIS CHAPTER

[What you need](#) / [Create this agent](#) / [The job and definition of done](#) / [Example inputs to enter](#) / [Follow the steps](#) / [Check the finished result](#)

Before following this example

Illustrative prompts to copy and adapt - replace you@gmail.com with your exact account

What you need

- A configured model connection and the exact Gmail account you intend to review.
- A Google Desktop OAuth client JSON with Gmail API enabled and the intended account permitted to consent. OAuth is Google's sign-in and permission process.
- The app's Gmail helper prepared, plus permission to connect and approve that account for the agent's project.
- An understanding that the chosen model provider receives the authorized mail information used for the task.

Create this agent

Agent name: Unread mail assistant

Copy or adapt these reusable agent instructions:

```
Prepare concise mail briefs. Prioritize observed deadlines and direct questions.
Separate explicit message content from guesses about urgency. Recommend actions
for me to take without sending or changing mail. State the number of messages
reviewed and every coverage or truncation limit.
```

The job and definition of done

Example objective:

```
Prepare a brief from the unread headers and snippets returned for my approved Gmail account. Prioritize deadlines in the next seven days, direct questions that need my reply, and payment-related messages requiring my attention. For each important item, show the sender, subject, observed deadline if present, and a suggested next action. Use headers and snippets only; do not read full threads or attachments.
```

What a useful completed result should contain:

A private unread-mail brief with important observed items and suggested next actions, plus returned message count, read limit, excluded unread messages and any shortened summaries.

Example inputs to enter

Use the fields named in the steps below. These are example values to adapt, not an automatically configured or completed task.

Your exact Gmail address:

```
you@gmail.com
```

Focus / review question:

```
Prioritize deadlines in the next seven days, direct questions that need my reply, and payment-related messages requiring attention. Show sender, subject, any observed deadline and a suggested next action. Use headers and snippets only. Do not read full threads or attachments. State the number reviewed, excluded unread messages and any shortened summaries.
```

Follow the steps

01 Create the mail agent

Choose the + beside YOUR AGENTS. Enter Unread mail assistant in Agent name, copy the example agent instructions into Instructions, and choose Create agent. A new agent starts without your mail connection. Its instructions guide its behavior, but account access is separately approved for the task and project.

02 Prepare the Gmail workflow

Open Workflows > Review unread Gmail > Use workflow. In Agent, select Unread mail assistant. In Gmail account, type your exact email address, replacing the illustrative you@gmail.com. Copy the example focus value into What matters to you?. The address saved on this task must match the account returned by Google consent.

03 Keep this task at snippet-level access

Choose a configured Model, review Maximum model spend (USD) and Advanced run limits, then choose Review task. In the reviewed form, leave Allow this task to search mail, read selected full threads, and import selected safe attachments (read only) unchecked. This optional checkbox expands detailed read access; leaving it clear keeps this example limited to the unread listing's headers and snippets.

04 Save the task before connecting

Read the task brief, read-only mail account and desired result, then choose Save task. The Gmail connection requires a saved live task naming the account, so this step comes before Connect Gmail. Saving makes no model calls and does not read the inbox.

05 Prepare the Google consent application

If you have not already done so, enable Gmail API for your Google Cloud project. Create a Desktop app OAuth client and download its JSON. The app's Setup instructions explain the testing setup, intended test user, branding and Gmail read-only scope. This Google configuration is an owner setup task; the agent does not create your Google Cloud project or credentials.

06 Connect the exact account

Open Settings > Gmail connection. If several saved mail tasks exist, choose the correct Saved Gmail task. Choose Import OAuth client JSON, select the downloaded client file, and choose Connect Gmail. Complete Google sign-in and consent in the system browser that opens. Return to the app and check Connected account. Use Refresh if needed. Enter passwords and verification codes in Google's page, never in the task conversation.

07 Approve the connection for the project

Choose Verify account for project use. Open Projects, select the project containing Unread mail assistant, and find Approved accounts > Gmail. Choose Approve followed by the verified account. Connection and project approval are separate steps: consent saves the local connection, while project approval allows this project to use that exact account. Recheck the address before approving.

08 Start the brief and handle waiting

Return to the saved task, open Activity and use Check again in Ready for this job?. Choose Run agent when account and model setup are ready. If Google consent expires or a different account is connected, resolve the connection request for this exact account. Answer clarification in the request card. A connection action does not override a task that you have paused.

09 Review the coverage with the brief

Open Results when the brief is ready. Review the actual items and suggested actions along with the returned count, per-read limit, excluded unread messages and shortening disclosures. Choose Accept this version or Request changes. Reply to messages yourself in Gmail if appropriate; the saved brief is a report, not a sent reply.

Check the finished result

- Does the result identify the same Gmail account you approved?
- Does every urgency statement come from observed headers or snippets, with guesses identified?
- Does the report disclose the up-to-50 unread-message read limit and whether more messages were excluded?
- Does it avoid claiming to have read full bodies, attachments or the complete inbox?
- Are suggested replies or actions clearly suggestions for you to carry out?

Scope and limitations

This example does not send, archive, delete or mark mail read. The baseline unread read returns up to 50 messages and can shorten summaries; unread status is preserved. Detailed thread and safe-attachment access is a separate opt-in and remains bounded. No browser-extension setup is required for Gmail API reads: Google consent uses your system browser, and the agent reads through the supported read-only connection.

CHAPTER 20 / STEP-BY-STEP EXAMPLES

Example 4 - Set up an agent to review supplied code

Use this when you want concrete findings about source files you explicitly provide. The example reviews a small JavaScript price calculator. The agent has no automatic access to your repository or other Mac folders.

IN THIS CHAPTER

[What you need](#) / [Create this agent](#) / [The job and definition of done](#) / [Example inputs to enter](#) / [Follow the steps](#) / [Check the finished result](#)

Before following this example

Illustrative prompts to copy and adapt - substitute your real source filenames

What you need

- A configured model connection.
- The source files and relevant context. For this example: price-calculator.js, price-calculator.test.js, README.md and test-fixtures.json.
- Small synthetic test inputs and instructions for any test command. Remove secrets or unrelated private data before supplying files.
- Docker and the prepared isolated code runtime if you want tests executed. Text review can still be useful when execution is unavailable, provided that limitation is disclosed.

Create this agent

Agent name: Code review assistant

Copy or adapt these reusable agent instructions:

```
Report concrete bugs tied to supplied filenames and line references when available. Explain a reproduction or failing input for each finding. Separate observed issues, proposed fixes and tests actually executed. Ask for missing context instead of assuming access to an entire repository. Keep reports and proposed fixes private.
```

The job and definition of done

Example objective:

```
Review the supplied JavaScript price calculator for incorrect totals, input validation, rounding and failure handling. Use the supplied fixtures and tests when possible. Prioritize findings by impact, explain a triggering input, propose a fix, and list checks actually run. Do not claim full-repository coverage.
```

What a useful completed result should contain:

One private report.md containing prioritized source-grounded findings, suggested fixes, file coverage, test evidence and limits, with unexecuted tests clearly marked.

Example inputs to enter

Use the fields named in the steps below. These are example values to adapt, not an automatically configured or completed task.

Focus / review question:

```
Review the supplied JavaScript price calculator for incorrect totals, input validation, rounding and failure handling. For each concrete finding, name the file, explain a triggering input and impact, and propose a fix. Use the supplied tests and fixtures if the offline runtime supports them. Distinguish tests actually executed from proposed tests. Do not claim full-repository coverage.
```

What the files represent:

```
price-calculator.js, price-calculator.test.js, README.md with intended behavior and test instructions, and test-fixtures.json. Request clarification if any required context is missing.
```

Follow the steps

01 Create the reviewer

Choose the + beside YOUR AGENTS. Enter Code review assistant in Agent name and copy the example agent instructions into Instructions. Choose Create agent. This establishes the reviewer's general habits. The individual task names the code and concerns for this review.

02 Choose the source-review workflow

Open Workflows > Review code files > Use workflow. Select Code review assistant in Agent. Copy the example focus value into What should the review focus on? and the example files value into Files or test fixtures to request. Replace the sample filenames and concerns with your real ones. For example, a Python review should describe the Python runtime and intended behavior instead.

03 Select the model and review access

Select your configured Model and review Maximum model spend (USD) plus Advanced run limits. Choose Review task. Confirm that no websites are granted for this file-only example and the task uses its private workspace. The workflow permits supplied file review and optional isolated offline testing; it does not grant repository checkout or arbitrary host-file access.

04 Save, then attach the source

Choose Save task. Open the saved task's Files panel > Import files and select the source, context and fixtures. Source-text extensions such as .js, .ts and .py are handled as text; .md and .json are also supported. Check the Inputs list. This workflow does not have the two named CSV assignment controls; the attached exact input versions provide its file context.

05 Decide what testing is possible

Open Activity > Ready for this job? and check the optional code-runtime status. A plain JavaScript test using Node's standard library can fit the prepared offline image. Tests needing a web server, network service or absent package may not. Explain expected behavior and runnable commands in the supplied README. Do not assume that attaching a lockfile makes all dependencies available.

06 Start the review

Choose Run agent when model setup is ready and you are satisfied with the supplied inputs. Watch the conversation and Activity. The agent may request missing source or fixtures by name. Respond through the request card, selecting the requested exact versions or importing replacements. If it requests a package, inspect the explicit dependency request rather than expecting automatic installation.

07 Keep unsupported testing visible

If the required runtime cannot run a test, decide whether a text-only review is acceptable. Answer a clarification or a proposed revised outcome explicitly. A proposed test is useful evidence for a next step, but it is not an executed test. Pause or Stop the task if you need to change the scope before more work.

08 Review findings and proposed fixes

Open Results and compare the findings with the supplied code. Check that line references, triggering inputs and executed checks support the priority given to each issue. Use Accept this version or Request changes. This example produces a private report and may include proposed fix files; applying a change to your real repository is a separate action you carry out or explicitly arrange.

Check the finished result

- Is each finding tied to code that was actually supplied, with a concrete triggering condition?
- Are intended behavior and missing context reflected in the conclusion?
- Does the report name files reviewed and files not provided?
- Are test commands, outcomes and execution limits recorded accurately?
- Does it distinguish a suggested fix from a fix applied to your actual project?

Scope and limitations

The agent reviews selected file snapshots. It cannot establish the behavior of the entire application from a few files. Offline tests cannot call production services or download arbitrary packages. A file review is not a complete security, accessibility or performance certification.

CHAPTER 21 / STEP-BY-STEP EXAMPLES

Example 5 - Set up a website usability review agent

Use this when you want a review of visible page content, clarity and consistency. This example looks at beginner information on Python's public website; it does not perform a full customer journey or modify the site.

IN THIS CHAPTER

[What you need](#) / [Create this agent](#) / [The job and definition of done](#) / [Example inputs to enter](#) / [Follow the steps](#) / [Check the finished result](#)

Before following this example

Illustrative prompts to copy and adapt - no site review is already completed

What you need

- A configured model connection.
- A short list of public HTTPS pages and a clear audience or review goal.
- The agent's dedicated Chrome profile and extension connected, or another supported browser backend already prepared.

Create this agent

Agent name: Website clarity reviewer

Copy or adapt these reusable agent instructions:

```
Review visible pages from the stated audience's perspective. Explain each observed problem with its page and evidence. Separate observed issues from suggestions and untested questions. Do not claim interaction, accessibility or performance testing unless those checks were actually executed. Keep the report private.
```

The job and definition of done

Example objective:

```
Review the Python homepage and Getting Started page for a person who has never programmed. Focus on the clarity of the first next step, explanation of unfamiliar terms, consistency between the pages and information a beginner may need before downloading or studying. Do not submit forms, create accounts or change website data.
```

What a useful completed result should contain:

One private report.md with page-specific observations, a prioritized improvement list, exact pages inspected, suggestions clearly labeled and unexecuted checks disclosed.

Example inputs to enter

Use the fields named in the steps below. These are example values to adapt, not an automatically configured or completed task.

Focus / review question:

```
The audience is a person who has never programmed. Review the clarity of the first next step, explanation of unfamiliar terms, consistency between the pages, and information needed before downloading or studying Python. Give page-specific evidence and practical improvements. Separate observed issues from suggestions. Do not claim full interactive, accessibility or performance testing.
```

Approved pages, one per line:

```
https://www.python.org/  
https://www.python.org/about/gettingstarted/
```

Follow the steps

01 Create the reviewer agent

Choose the + beside YOUR AGENTS. Enter Website clarity reviewer in Agent name and copy the example agent instructions into Instructions. Choose Create agent. These instructions define a careful review style; the task will define which audience and pages to inspect.

02 Choose the website workflow

Open Workflows > Review a website > Use workflow. In Agent, select Website clarity reviewer. Copy the example focus value into Audience and review goals and the example websites value into Pages to inspect. Replace them with your own website and audience if desired. Use one complete public HTTPS link per line.

03 Make the goal small enough to review

Name a particular audience and a few pages, as the example does. "Review everything" is hard to verify and may exceed the budget. If you want a purchase journey, specify the public pages you can authorize and recognize that this read-only workflow does not complete checkout or submit forms. Start with visible information you can assess in a report.

04 Choose a model, review and save

Choose your configured Model. Review Maximum model spend (USD) and Advanced run limits. Choose Review task and confirm the brief, outcome and read-only allowed websites. Choose Save task. No model work begins yet. The saved task retains this reviewed scope.

05 Prepare the browser

Open the saved task's Browser panel > Browser setup. For Desktop Chrome · dedicated profile, follow Open Chrome setup, Developer mode and Load unpacked if needed, selecting the app-shown extension folder. Use Check extension afterward. Open Activity and Check again in Ready for this job? to resolve setup issues before the run.

06 Run the visible-page review

Choose Run agent. Follow the conversation and Activity as it inspects the approved pages. If a page cannot be read, answer the clarification request with an allowed substitute or accept an explicitly reduced scope. If owner sign-in is needed, choose Take control in Chrome, complete it on the website, then Return to agent. Credentials do not belong in the task message.

07 Distinguish evidence from interpretation

A report may identify confusing language or an unclear link using an observation of a page. It should label inferred user difficulties as interpretation. If no screen-reader audit, load-time measurement or completed form interaction was executed, those checks must remain untested. Use Pause if you want to redirect the agent before it continues.

08 Review and choose a next step

Open Results when complete. Check its page list and prioritize the observed issues you can reproduce. Choose Accept this version or Request changes for a more focused second pass. PDF report and Word document are available for supported complete text reports. A report's acceptance does not change the website.

Check the finished result

- Does each observed issue identify its page and the visible evidence?
- Does the review use the audience you specified rather than an assumed expert audience?
- Are suggestions distinguished from problems actually observed?
- Are inaccessible pages and unexecuted interaction, accessibility and performance checks listed?
- Can you reproduce the higher-priority findings before making changes?

Scope and limitations

The built-in workflow uses read-only approved browsing. It is useful for visible content and usability observations, with partial coverage disclosed. It does not replace a complete accessibility audit, instrumented performance test, real user study or authorized interactive workflow.

CHAPTER 22 / STEP-BY-STEP EXAMPLES

Example 6 - Set up a research agent for a decision

Use this when you want options compared against your own constraints. This example chooses a starting path for learning Python using sources you approve. The report remains a recommendation for your review.

IN THIS CHAPTER

[What you need](#) / [Create this agent](#) / [The job and definition of done](#) / [Example inputs to enter](#) / [Follow the steps](#) / [Check the finished result](#)

Before following this example

Illustrative prompts to copy and adapt - the example does not claim a recommendation has been verified

What you need

- A configured model connection.
- Your decision, preferences and constraints written down.
- Public HTTPS sources for the options, plus the agent's browser setup completed.

Create this agent

Agent name: Decision research assistant

Copy or adapt these reusable agent instructions:

```
Compare options against the owner's stated criteria. Cite inspected sources and source dates where available. Make missing information and assumptions visible. Explain trade-offs and the reason for a recommendation. Ask a focused question when a missing preference would materially change the answer.
```

The job and definition of done

Example objective:

```
Help me choose a first learning path for Python. I am a complete beginner, can study five hours per week, and prefer an option I can start without paying. Compare Harvard CS50's Introduction to Programming with Python and the official Python tutorial using the approved pages. Consider prerequisites, structure, practice exercises and the likely first steps. Distinguish published facts from your estimate of beginner difficulty.
```

What a useful completed result should contain:

One private report.md with a criteria-based comparison, a reasoned recommendation, first steps, exact sources and dates where available, missing information and uncertainty.

Example inputs to enter

Use the fields named in the steps below. These are example values to adapt, not an automatically configured or completed task.

Analysis question:

```
Help me choose a first learning path for Python. I am a complete beginner, can study five hours per week, and prefer an option I can start without paying. Compare Harvard CS50's Introduction to Programming with Python and the official Python tutorial. Consider prerequisites, structure, practice exercises and first steps. Separate published facts from estimated beginner difficulty. State unknowns before recommending a path.
```

Approved pages, one per line:

```
https://cs50.harvard.edu/python/  
https://docs.python.org/3/tutorial/index.html
```

Follow the steps

01 Create an agent with a decision-making style

Choose the + beside YOUR AGENTS. Enter Decision research assistant in Agent name and copy the example agent instructions into Instructions. Choose Create agent. Its ongoing role is to compare against your criteria; a task will supply the particular decision and source access.

02 Choose the decision workflow

Open Workflows > Research a decision > Use workflow. Select Decision research assistant in Agent. Copy the example question value into Decision, preferences and constraints and the example websites value into Sources the agent may use. Replace those values with your actual decision. Include budget, available time and must-have requirements so the recommendation can be judged.

03 Choose sources deliberately

The two example links provide starting information for the two learning paths. A task can inspect approved websites, but it does not automatically search the entire web. Add other relevant public HTTPS sources before saving if your question needs them. Where facts may change, such as pricing or prerequisites, ask the report to show source dates and information it could not confirm.

04 Choose the model and set limits

Choose a configured connection in Model and review Maximum model spend (USD). Open Advanced run limits and review model calls, tool steps, active seconds and token allowance. Use a small reviewed trial for a narrow question. A connection being configured does not prove its server is currently available or its model is compatible.

05 Review and save the brief

Choose Review task. Confirm the decision, constraints, expected result and allowed websites. Choose Save task. This creates a saved task without starting it. The workflow's generated brief and completion criteria become the task; there is no separate purchase or enrollment action.

06 Finish browser setup and run

Open Browser > Browser setup for this agent and connect its dedicated Chrome extension if needed. In Activity, use Check again in Ready for this job?. Choose Run agent once setup is ready. If the approved pages cannot support the requested comparison, the agent should request clarification or show the missing facts.

07 Answer questions that affect the choice

If the agent asks whether you prefer videos, reading or formal assignments, answer in the request card. New direction in the task conversation is included at the next agent decision. If you want to stop the attempt, use Pause or Stop. A missing preference is not permission to invent one.

08 Assess the recommendation against your constraints

Open Results and read the comparison, sources, uncertainties and proposed first steps. Check that the recommendation follows the criteria you typed. Use Accept this version or Request changes to prepare a narrower revision. Verify material unknowns yourself before paying for, enrolling in or otherwise committing to an option.

Check the finished result

- Are the compared options assessed against your stated budget, time and preferences?
- Are costs, prerequisites and availability tied to inspected sources rather than assumed?
- Are source dates, unknowns and conflicting information disclosed?
- Does the explanation show why the recommended option fits, including trade-offs?
- Are predicted difficulty or completion time identified as estimates?

Scope and limitations

A recommendation can only be as well supported as the sources and preferences supplied. The agent does not enroll, purchase or commit on your behalf. Pages and product details can change; the report must disclose dated information and unconfirmed claims.

CHAPTER 23 / STEP-BY-STEP EXAMPLES

Example 7 - Establish a fleet to review published files

Use this when one objective benefits from several agents reviewing the same evidence. A fleet is a bounded team: its lead proposes roles and work, specialists claim tasks, and the lead combines their reports. You choose the objective, models, source versions and limits.

IN THIS CHAPTER

[What you need](#) / [Create this agent](#) / [The job and definition of done](#) / [Example inputs to enter](#) / [Follow the steps](#) / [Check the finished result](#)

Before following this example

Illustrative prompts to copy and adapt - this describes a possible fleet, not a completed live assessment

What you need

- Configured model connections for Lead model and Worker model. They may use the same configured connection.
- A project to keep the evidence and team together.
- Supported published text evidence. This example uses launch-brief.md, pricing-policy.md and support-notes.md with fictional or shareable content.
- No more than eight selected evidence files, up to 64 KiB each and 256 KiB in total. KiB is a file-size unit; 64 KiB is about 65,000 bytes.

Create this agent

Agent name: Launch evidence organizer

Copy or adapt these reusable agent instructions:

```
Keep imported launch evidence private until the owner publishes an exact version.  
Do not claim the documents establish live website behavior. Describe missing  
evidence and keep source versions identifiable.
```

Agent setup distinction

This manually created agent holds the private evidence before publication. Its instructions do not become fleet instructions. The fleet creates its own lead and specialists from the objective; there is no manual Agent instructions field in the Fleets form.

The job and definition of done

Example objective:

```
Review the selected launch documents for contradictions and missing information  
that could confuse a first customer. Organize distinct reviews of offer clarity,  
pricing consistency and support expectations; compare findings against the  
selected evidence, exchange findings and combine the reports. Prioritize issues  
with exact document evidence, recommended wording changes and questions still  
requiring verification. State that this is a file-only review and no live  
website, payment flow or customer interaction was tested.
```

What a useful completed result should contain:

One combined report that traces prioritized findings to selected evidence and specialist reports, resolves or discloses disagreements, proposes changes and names the live checks that remain untested.

Example inputs to enter

Use the fields named in the steps below. These are example values to adapt, not an automatically configured or completed task.

New project name:

```
Launch review
```

Project description:

```
A document-only review of fictional or approved launch evidence.
```

Evidence-preparer task objective:

```
Hold the supplied launch documents for my review and explicit publication. Do not  
begin analysis until I choose to run a task.
```

What done looks like for the preparation task:

```
The supplied documents are imported privately and identifiable by exact version.
```

Fleet mode:

```
Review published files
```

Example evidence filenames - supply your own copies:

```
launch-brief.md  
pricing-policy.md  
support-notes.md
```

Follow the steps

01 Create a project for the review

Open Projects. Under New project, enter Launch review in Name and the example description in Description, then choose Create project. A project groups its agents, approved shared references and collaboration. Existing agents cannot be moved between projects, so create the evidence organizer inside this project.

02 Create the private evidence organizer

In the selected project's Add an agent form, enter Launch evidence organizer in Agent name and copy the example agent instructions into Agent instructions. Choose Create agent in Launch review. The files will initially stay private to this agent. This is a convenient way to inspect what you intend to share with the future team.

03 Save an input-holding task

Beside this project agent, choose New task. Select Live agent. Copy preparationObjective into What should the agent accomplish? and preparationDoneLooksLike into What does done look like?. Choose a configured Model, review the limits, keep Task access at Private workspace and approved browser tools and Allowed websites empty, then choose Save live task. Leave this preparation task paused; you do not need model calls merely to import and publish owner-selected files.

04 Import and inspect the evidence

Open the saved preparation task's Files panel > Import files and select the three example documents or your own equivalents. Inspect each saved file before sharing it. Supply compact text, Markdown, JSON or CSV evidence; images, PDFs and spreadsheet workbooks are not eligible inputs for this fleet mode. Make sure the content fits the file and total-size limits.

05 Publish the exact versions inside the project

Open each private file's details, choose Publish, read the sharing explanation, and choose Publish within project. Publishing creates an immutable shared version: other agents in this project can discover and use it. It does not publish a webpage on the internet. A later version will not replace the one the fleet selects.

06 Prepare the fleet objective and sources

Open Fleets and expand Start a fleet if necessary. In How should the fleet work?, choose Review published files. Select Launch review in Project. Copy the example objective into What should the team accomplish?. Under Evidence for this fleet, select the exact three published versions. Use Open shared library if the sources are missing and check that they belong to the same project.

07 Choose lead, worker and team ceilings

Select configured connections in Lead model and Worker model. The lead plans and combines results; the worker connection is used for specialists the lead creates. Review Total model spend limit (USD), then expand Team size and limits. Review Agents, including the lead; Tasks working at once; Work items; Plan revisions; call, token and combined-time ceilings; and Limits for each task. A workspace concurrency setting may reduce simultaneous work.

08 Save a draft, then deliberately start

Choose Save draft to preserve the reviewed objective, evidence and limits without model calls. In Your fleets, inspect the draft and choose Start fleet when ready. Alternatively, Start fleet on the creation form saves and starts in one action. One start includes planning, specialist tasks and automatic handoffs within your chosen limits; the lead does not ask for approval at every handoff.

09 Follow the team and resolve blockers

Read Team, Shared work board, Plan revisions and Findings and messages. Open task shows an individual agent's conversation and Activity; Read result opens a completed specialist report. Answer clarification requests in their task cards. Use Pause fleet to retain progress or Stop fleet to end execution. Resume fleet continues saved work under its existing models and budgets; it does not reset the allowance.

10 Review the combined evidence

If the fleet completes, choose Open combined result. Review the final report against selected evidence, exact specialist reports and successful tool receipts. Use Accept this version when satisfied. The fleet result does not use the ordinary Request changes revision flow; for another pass, start a new fleet selecting the evidence and reports you want reviewed.

Check the finished result

- Are the selected project and exact file versions correct?
- Did the team divide work meaningfully and support findings with document evidence?
- Are specialist reports and disagreements reflected in the combined report?
- Does every claimed live test remain explicitly untested in this file-only mode?
- Are budget-limited or incomplete work items visible before accepting the result?

Scope and limitations

Review published files has no live browser, network target or code execution. It reviews bounded selected text evidence and fleet reports. Publicly documented integration checks demonstrate coordination mechanics with scripted models; they do not establish the quality of a hosted model's planning or every real fleet run. The team may stop or need attention before producing a combined result.

CHAPTER 24 / STEP-BY-STEP EXAMPLES

Example 8 - Establish a fleet in the local website lab

Use this as a practice environment for agents exploring a website together. Harbor Desk is a fictional website owned by the app. The fleet starts as logged-out visitors with the site address and discovers pages through its browser.

IN THIS CHAPTER

[What you need](#) / [Create this agent](#) / [The job and definition of done](#) / [Example inputs to enter](#) / [Follow the steps](#) / [Check the finished result](#)

Before following this example

Illustrative objective to copy - the live demo remains incomplete and this is not a promised successful run

What you need

- The desktop app opened and configured Lead model and Worker model connections.
- A project for the practice run.
- The app's training website able to start at <http://127.0.0.1:4318/>. Another process using that port blocks setup.
- Docker and the prepared code runtime only if the fleet needs optional offline calculations over permitted reports. The lab browser itself needs no Chrome extension or Docker.
- A reviewed budget: live cloud-model calls can cost money. The repository's paid live lab assessment is paused and incomplete.

Create this agent

Agent name: Fleet-generated lead and specialists

Copy or adapt these reusable agent instructions:

Work only on the app's fictional Harbor Desk website. Use synthetic accounts and invented data. Report only behavior you actually observe, include steps and evidence for each finding, and disclose any journey you could not complete.

Agent setup distinction

Do not look for a manual agent-name or Agent instructions field in this mode. The fleet creates its team. The guidance above is already included in the copyable fleet objective below.

The job and definition of done

Example objective:

```
Explore the app's fictional Harbor Desk website as a new visitor. Organize a bounded team to review public information, self-registration if offered, and the user journeys available to ordinary synthetic accounts. Work only on this local training website and use invented account data. Exchange findings, reproduce unexpected behavior when possible, and deliver one prioritized report with steps, observed evidence, impact and suggested remediation. Do not claim a check passed or a flaw was found without observing it. Disclose incomplete journeys and all coverage limits.
```

What a useful completed result should contain:

If the run completes, one combined report grounded in real lab observations and specialist reports, with reproducible steps for observed findings and an honest list of unfinished checks.

Example inputs to enter

Use the fields named in the steps below. These are example values to adapt, not an automatically configured or completed task.

New project name:

```
Harbor Desk practice
```

Project description:

```
A bounded practice run on the app's fictional local website.
```

Fleet mode:

```
Test the local website
```

Follow the steps

01 Understand the practice boundary

Harbor Desk is a synthetic site, so this example concerns invented data and ordinary visitor or self-created synthetic accounts. It is separate from a public-site review. As of the documented 1 October 2026 status, the paid live fleet assessment is paused and incomplete: it has no final report, verified model-discovered flaw or demonstrated simultaneous live specialist work. Use this tutorial as a practice setup, not proof that such a run has already succeeded.

02 Create a practice project

Open Projects > New project. Enter Harbor Desk practice in Name and the example project description in Description, then choose Create project. You do not need to manually create the fleet's agents: the team lead and specialists are created by the fleet within the saved agent limit.

03 Choose the lab mode and write the objective

Open Fleets and expand Start a fleet. In How should the fleet work?, choose Test the local website. Select Harbor Desk practice in Project. Copy the example objective into What should the team accomplish?. The objective includes the working rules; there is no separate Agent instructions field on this form.

04 Start the app-owned training site

Under Local training website, choose Start training website. Check that the app reports Website ready and shows <http://127.0.0.1:4318/>. The app verifies that this is its own site. If another service owns the port or startup fails, resolve the setup issue; the app does not adopt an unrelated website as the lab.

05 Choose models and bound the exercise

Choose configured connections in Lead model and Worker model. Review Total model spend limit (USD), expand Team size and limits, and inspect both team and per-task ceilings. For a first exercise, keep the objective narrow and the team small. Setting Tasks working at once to 1 is a valid way to make the activity easier to follow; choose up to 2 only if you want and have capacity for overlapping work.

06 Save the draft before running

Choose Save draft to save without model calls. Read the saved objective, models and limits. Choose Start fleet when the training site and connections are ready. A start authorizes planning and automatic handoffs within those limits. Saving a draft can succeed before site or connection readiness; it does not verify a complete live run.

07 Observe the separate agent sessions

Use Team or Shared work board > Open task to inspect each agent. Its Browser panel shows Local lab Electron · Temporary session. Agents begin logged out, without privileged credentials or source files. Each has separate cookies and can register an invented account if the site offers registration. Browser observations and bounded local requests become evidence; ordinary Mac browsing stays separate.

08 Use owner controls when needed

Answer clarification in the relevant task's request card. In the lab browser, Take control in local window gives you exclusive control; Return to agent gives it back. Use Pause fleet to retain task progress and close lab browsers, or Stop fleet to end execution. Closing a lab browser clears its cookies and storage, so reopening starts a new visitor session. Register a fresh synthetic account; the lab has no returning-account sign-in route.

09 Review only demonstrated outcomes

If a final result is produced, choose Open combined result. Compare the claimed findings with observations, receipt records and reproduction steps. Review which journeys remained unfinished or could not be checked within the limits. Choose Accept this version only after assessing those limits. If the fleet needs attention or ends early, keep that outcome visible instead of treating it as a completed assessment.

Check the finished result

- Did the agents stay within the app-owned fictional site and use synthetic data?
- Does each reported issue have observed evidence and clear reproduction steps?
- Are specialist claims, report versions and successful receipts traceable?
- Are incomplete journeys, closed-session login loss and budget limits disclosed?
- Does the report avoid turning untested possibilities into verified flaws or passing checks?

Scope and limitations

The lab has a fixed local target, a separate temporary browser per agent, bounded local HTTP checks and optional offline code over allowed fleet reports. It is not an unrestricted terminal, general PC controller or arbitrary public-site testing system. The real live assessment remains incomplete, and the final navigation-readiness change still requires its dedicated real-browser verification described in the repository status.

CHAPTER 25 / REFERENCE

Further reading and developer commands

Use these sources when you need the implementation detail behind the guide.

IN THIS CHAPTER

[Repository documentation](#) / [Developer verification](#) / [Local packaging](#) / [Guide edition](#) / [Your first-session checklist](#)

This guide describes the implemented local pre-release. Original planning documents contain proposals and provisional defaults. Read the architecture, design, and implementation plan in that order. design-preview.html is an interactive planning mockup, not the working app.

Repository documentation

- [Source repository and README](#)
- [Original architecture baseline](#)
- [Original product design](#)
- [Original implementation plan](#)
- [Model-provider contract and limitations](#)

- [Fleet website lab: implementation and incomplete live-run evidence](#)
- [Portable code setup and backup/recovery foundations](#)

Developer verification

These commands are for source maintainers. Default Electron tests use temporary data, scripted models, and synthetic fixtures. Separate opt-in tests can require Docker, Chrome, Keychain, or paid provider calls; read their scope before enabling them.

```
npm run typecheck
npm run build
npm run test:electron
npm run test:fleet:electron
```

Local packaging

Build all native helpers first, then validate the package inventory. `npm run package:mac` produces an Apple Silicon local application and manifest in a new directory. It is ad-hoc signed, not a public notarized installer. It includes neither models/accounts nor Docker images.

```
node scripts/package-mac.mjs --check
npm run package:mac
```

Guide edition

Prepared 2 October 2026 for version 0.10.0. The online page and PDF share the same guide content. Examples are instructional prompts and fictional data, not claims of completed live runs. Confirm UI/source changes against the current repository before following a later release.

Your first-session checklist

- I can open the local desktop app and recognize its current Agent Workspaces name.
- I understand which project the work belongs to and have created an agent with useful general instructions.
- I know whether my selected model runs locally or sends task content to a cloud provider.
- I chose an example workflow, adapted its question, and reviewed the prepared objective and definition of done.
- I assigned the required file versions or approved websites/account. I resolved the readiness items that matter for this job.
- I reviewed model spend, calls, tokens, tool steps, and active-time limits before deciding to run live work.
- I know where to answer requests, take browser control, pause, and stop. I will enter credentials only through the proper settings or sign-in flow.
- I will read the exact output, evidence, and coverage before accepting it. I will keep any private backup and exported data safe.

A good first attempt

Choose a small harmless job with inputs you understand. A blocked request or clearly reported limitation is useful information. Improve the brief and setup based on observed results; do not judge success only by whether the app reaches Complete.